
ECE 285 Final Report

Yu-Shu Chen

Department of Electrical and Computer Engineering
University of California San Diego
La Jolla, CA 92093
yuc067@ucsd.edu

Abstract

We study the reliable memorization capacity of transformer-based language models on high-entropy, incompressible sequences. We generate synthetic datasets of random sequences of length 10 over a 8-token vocabulary and measure the maximum number of sequences a model can perfectly reproduce when conditioned on a prefix. Using a GPT-style decoder-only transformer, we systematically vary model depth, embedding width, and attention head count, training each configuration until full sequence reconstruction is achieved. We define *critical capacity* as the largest dataset size for which complete memorization occurs, providing a controlled measure of how transformers allocate parameters to store independent sequences, independent of statistical generalization. Our results reveal strong anisotropy in scaling: depth is a more reliable and parameter-efficient driver of memorization than width, while excessive embedding width ($d_{\text{model}} \geq 256$) can actively degrade capacity due to optimization bottlenecks. These findings establish empirical upper bounds on transformer storage efficiency and clarify the interplay between architecture, initialization, and parameter utilization.

1 Introduction

Large language models are trained to approximate probability distributions over high-dimensional discrete sequences. While their generalization capabilities have been extensively studied, significantly less is known about their reliable memorization capacity, namely, how many independent sequences can be stored in model parameters and reproduced perfectly under controlled conditions.

Recent work has examined factual memorization in language models, often focusing on structured natural language data such as subject–relation–object triples. However, natural language contains substantial redundancy and regularity, allowing models to exploit shared structure when storing information. As a result, memorization performance in such settings reflects a mixture of compression and storage, rather than pure capacity.

In this work, we isolate raw storage capacity by considering high-entropy, uniformly random discrete sequences. Each sequence is sampled independently from a fixed vocabulary, minimizing compressibility and shared structure. In this setup, successful reproduction of training sequences cannot rely on statistical regularities and instead reflects the model’s ability to allocate parameter capacity toward storing individual instances.

We focus on autoregressive transformer-based language models and investigate the following question: *how many independent random sequences can a transformer reliably store and reproduce perfectly when conditioned on a prefix?*

To answer this, we systematically measure critical memorization capacity (N_{crit}) as the absolute maximum number of sequences for which a model achieves 100% exact reconstruction accuracy on the target payload.

By systematically varying both model architecture and dataset size under controlled, stochastic conditions, we empirically characterize how reliable storage scales with parameter count and topology.

Our approach provides a **quantitative map of transformer memorization**, evaluating the interplay between architectural depth, embedding width, total parameter count, and optimization dynamics, while explicitly accounting for stochastic variance due to random initialization. This allows us to identify structural and optimization factors that govern realized capacity and establish empirical upper bounds on memorization across architectures.

Our core findings are as follows:

- **Anisotropic Scaling and Parameter Efficiency:** Total parameter count alone is a poor predictor of memorization capacity. Storage efficiency is heavily dictated by architectural shape, with scaling network depth providing a more reliable and parameter-efficient mechanism for prefix–payload routing than scaling width.
- **Width Optimization Bottleneck:** Across a comprehensive 2D architectural grid, excessively widening the embedding dimension ($d_{\text{model}} \geq 256$) degrades realized capacity, revealing a divergence between theoretical storage limits and achievable empirical performance.

All code and dataset generation scripts are available at <https://github.com/YuShuRChen/transformer-capacity>.

2 Related work

Neural associative memory. The storage capacity of neural systems has long been studied in the context of associative memory. Classical Hopfield networks (Hopfield [1982]) can reliably store approximately $0.14N$ random binary patterns for N neurons (Folli et al. [2017]), while extensions such as Dense Associative Memories improve scaling behavior (Krotov and Hopfield [2017]). These models provide explicit attractor dynamics and theoretical capacity bounds under random pattern storage assumptions.

Modern deep generative models, including transformers and latent-variable models, are not explicitly designed as associative memories. Nevertheless, they must implicitly store information about training examples within their parameters. Their reliable storage limits for random symbolic sequences, however, remain poorly characterized. In this work, we empirically investigate these limits under controlled random sequence storage tasks, providing a connection between classical memory-oriented architectures and contemporary generative models.

Memorization and scaling laws. Recent work has studied how large language models memorize facts, often framed as subject–relation–object triples. In Lu et al. [2024], they analyzed how LLMs store and retrieve structured facts from sources like Wikidata, showing that memorization scales roughly linearly with model size and decays exponentially with the number of training epochs. They also observe that LLMs preferentially memorize frequent or difficult facts, can generalize to unseen facts, and struggle to store redundant or conflicting information. These findings show that even very large models cannot reliably memorize all public facts under typical pre-training.

In this work, we use uniformly random discrete sequences, which are high-entropy and roughly incompressible. Each sequence carries almost independent information, minimizing shared structure. From an information-theoretic perspective, this setup isolates the model’s architectural storage capacity rather than its ability to compress patterns. In other words, we are measuring how much information the model can reliably store, providing a stress test of memorization close to the theoretical limits predicted by information theory.

3 Methodology

3.1 Problem formalization

Let $\mathcal{D} = \{s_1, s_2, \dots, s_N\}$ be a dataset of N sequences. Each sequence s_i consists of tokens $t \in \{1, \dots, V\}$ and is partitioned into a prefix of length L_p and a payload of length L_y , with total length $L = L_p + L_y$. The prefix serves as a unique key, and the payload is the information to be memorized.

We consider a decoder-only transformer f_θ with parameters θ . The objective is to learn θ such that, given the prefix, the model exactly reproduces the corresponding payload for every sequence:

$$\forall s \in \mathcal{D}, \quad \arg \max P(t_k \mid t_{1:k-1}; \theta) = t_k^{\text{true}}, \quad \forall k \in \{L_p + 1, \dots, L\}.$$

We define the *critical capacity* N_{crit} as the maximum N for which the model achieves 100% training accuracy on payload tokens.

3.2 Information-theoretic storage analysis

We quantify memorization efficiency by measuring the amount of target information stored per parameter. For a dataset $\mathcal{D}$ of N independent sequences, the model acts as an associative memory mapping prefixes (keys) to payloads (values) of length L_y over vocabulary size V .

Since prefixes are provided at inference time, we only count the information that must be generated. The total target entropy is:

$$H(\mathcal{D}) = N \cdot L_y \cdot \log_2(V).$$

If a model with P parameters achieves perfect payload reconstruction, we define the *storage efficiency*:

$$\eta = \frac{H(\mathcal{D})}{P} \quad (\text{bits per parameter}).$$

This is a strict lower bound on the true storage efficiency. In addition to the payload, the model must implicitly encode the prefixes and the mapping between prefixes and payloads. As a result, the total information stored in the parameters exceeds $H(\mathcal{D})$. Despite this, payload entropy provides a consistent baseline for comparing architectures.

4 Experimental setup

4.1 Dataset generation

We generate synthetic datasets of varying size N . Each sequence has length $L = 10$, with tokens sampled uniformly from a vocabulary of size $V = 8$.

This produces high-entropy, incompressible data, preventing the model from exploiting statistical structure. As a result, successful reconstruction requires explicit storage in the model parameters. This setup serves as a direct stress test of memorization capacity as a function of model size.

4.2 Model architecture

We use a GPT-style decoder-only Transformer with standard components: multi-head self-attention, feed-forward layers, layer normalization, and residual connections. Tokens are represented using learned embeddings with positional encodings.

To study architectural effects, we perform a grid search over width and depth while fixing the number of attention heads ($h = 4$):

- **Width** (d_{model}): $d_{\text{model}} \in \{32, 64, 128, 256\}$
- **Depth** (n_{layers}): $n_{\text{layers}} \in \{1, 2, 4, 6\}$

This yields 16 configurations. By relating critical capacity N_{crit} to total parameter count P , we evaluate how architectural shape (width vs. depth) affects storage efficiency.

4.2.1 Training procedure

We decouple single-model optimization from the global search over dataset size N .

1. **Single-run optimization:** For a fixed dataset, the objective is to achieve 100% accuracy on payload tokens. To isolate structural capacity from optimization artifacts, we use AdamW with the following settings:
 - *Zero weight decay:* prevents degradation of memorized sequences.
 - *Fixed batch size* ($B = 128$): independent of N . For small datasets, this introduces stochastic gradient noise, avoiding the stagnation typical of full-batch training.

- *Loss-driven early stopping*: convergence is determined using cross-entropy loss with a patience threshold. Monitoring loss rather than accuracy avoids premature termination when accuracy plateaus but confidence is still improving.
2. **Capacity search**: We treat training as a boolean evaluation (success/failure) and search for N_{crit} in two phases:
 - *Exponential search*: progressively doubles N to identify upper and lower bounds where convergence transitions to failure.
 - *Binary search*: refines the estimate within these bounds up to a relative tolerance ϵ .
 3. **Stochastic robustness**: Each configuration is evaluated over $N_{\text{trials}} = 16$ independent runs. We define capacity as the *maximum* N_{crit} observed across trials, capturing the best-case achievable memorization under favorable initialization.
- The number of trials is kept constant across configurations. Because the binary search uses a relative tolerance, the acceptable error window scales with N , keeping the difficulty of identifying N_{crit} approximately constant across model sizes.

5 Results

5.1 Isolating the capacity limit

To characterize the capacity boundary, we examine training accuracy as a function of dataset size N for a fixed architecture configuration across multiple random seeds. Figure 1 shows the evaluated points during the search procedure.

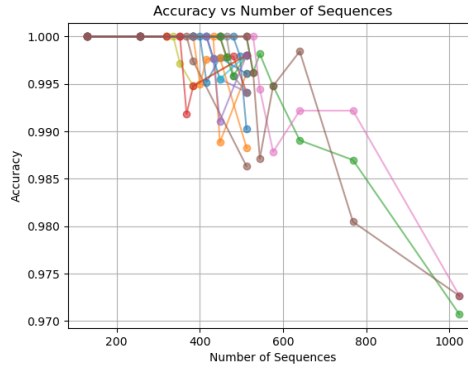


Figure 1: Training accuracy as a function of dataset size N for $d_{\text{model}} = 32$ and $n_{\text{layers}} = 6$

For N below the capacity limit, all trials consistently achieve 100% payload accuracy. As N increases, this behavior does not break down abruptly. Instead, there exists a narrow transition region where outcomes become seed-dependent: some runs still reach perfect accuracy, while others plateau slightly below (e.g., $\sim 99\%$). Beyond this region, no trial achieves perfect reconstruction, and accuracy consistently falls below 1.0. We take this point as evidence that the structural capacity N_{crit} has been exceeded.

5.2 Optimization Variance

Due to the non-convex loss landscape, we measure the variability of N_{crit} across 16 independent random initializations. Figures 2 and 2 show the resulting distribution for a representative architecture.

We observe substantial variance in capacity, ranging from approximately $N = 340$ to $N = 525$. This wide spread indicates strong sensitivity to initialization and data ordering. Notably, all runs remain stable during training; differences arise from whether optimization converges to a perfect memorization solution or plateaus slightly below it.

This variability motivates defining structural capacity as the *maximum* N_{crit} across trials. Mean or

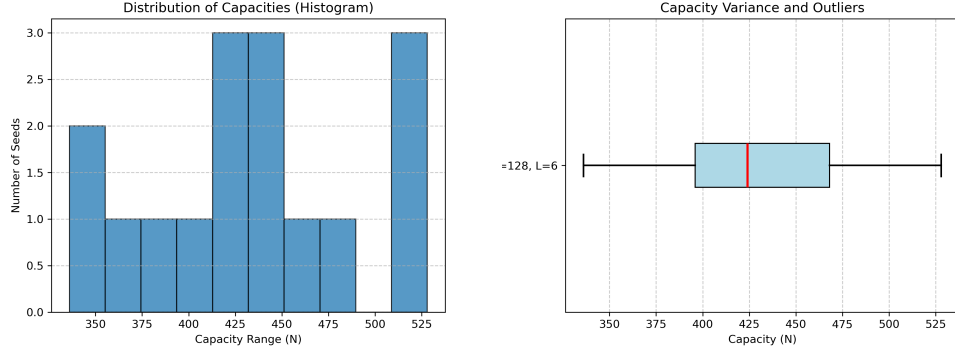


Figure 2: Distribution (left) and variance (right) of N_{crit} across 16 trials for $d_{\text{model}} = 32$ and $n_{\text{layers}} = 6$.

median estimates would conflate architectural limits with optimization outcomes. In contrast, the maximum isolates the highest achievable capacity under favorable conditions, providing an estimate of the model’s parameter-limited ceiling.

5.3 Architectural Scaling: Width vs. Depth

With the capacity metric established, we evaluate how embedding dimension (d_{model}) and depth (n_{layers}) affect N_{crit} .

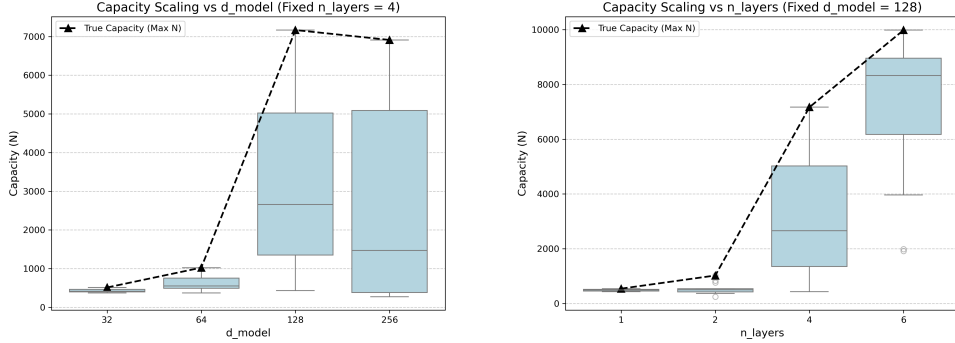


Figure 3: Capacity scaling trends. Left: Scaling d_{model} at fixed depth. Right: Scaling n_{layers} at fixed width. Dashed lines connect the maximum structural capacities.

At fixed width ($d_{\text{model}} = 128$), increasing depth yields a strictly monotonic increase in capacity. The scaling is non-linear, with a pronounced jump between $n_{\text{layers}} = 2$ and $n_{\text{layers}} = 4$. Although optimization variance increases with depth, the maximum achievable capacity continues to rise, reaching approximately $N \approx 10,000$ at $n_{\text{layers}} = 6$.

At fixed depth ($n_{\text{layers}} = 4$), scaling width reveals a different pattern. Capacity increases from $d_{\text{model}} = 32$ to 128, but does not continue improving at $d_{\text{model}} = 256$. Instead, the maximum capacity plateaus and slightly decreases. This is accompanied by a sharp increase in variance: while some runs remain competitive, the median performance drops substantially, and several seeds fail to reach high-accuracy solutions.

To examine joint effects, we plot N_{crit} across the full grid (Figure 4).

Capacity generally increases with both depth and width up to $d_{\text{model}} = 128$. However, at $d_{\text{model}} = 256$, performance degrades consistently across all depths, indicating a breakdown in effective scaling at larger widths.

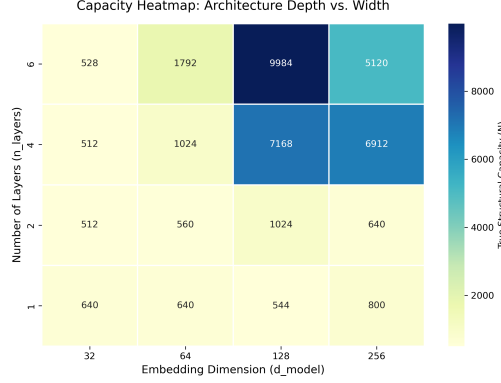


Figure 4: 2D interaction heatmap of structural capacity. An empirical optimization bottleneck is observed at the widest embedding dimension.

5.4 Parameter Efficiency and Scaling Laws

To test whether memorization capacity is determined primarily by total parameter count P , we aggregate all configurations into a log-log plot of N_{crit} versus P (Figure 5), with color indicating depth.

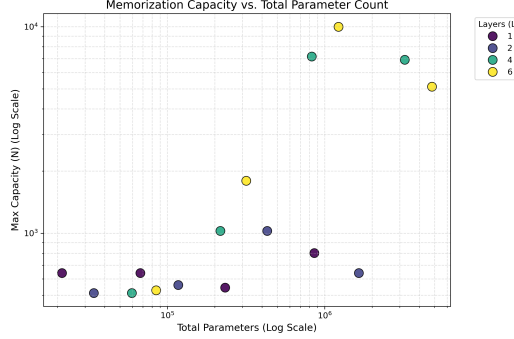


Figure 5: Log-log scaling of maximum memorization capacity against total trainable parameters.

If capacity scaled purely with parameter count, we would expect a tight linear trend on the log-log scale. Instead, we observe substantial vertical dispersion, indicating that models with similar parameter counts can achieve significantly different capacities.

This dispersion aligns with architectural differences. At a fixed budget of approximately 10^6 parameters, a 6-layer model achieves $N \approx 10,000$, while a 1-layer model with a comparable number of parameters remains below $N \approx 1,000$. Deeper models ($n_{\text{layers}} = 4, 6$) consistently occupy the upper envelope of the plot, achieving higher capacity per parameter than shallower configurations.

We also observe that increasing parameter count alone does not guarantee higher capacity. The largest models (rightmost region of the plot) do not extend the observed capacity frontier and, in some cases, exhibit reduced maximum capacity. This is consistent with the optimization limitations observed in the width scaling experiments.

Overall, these results indicate that parameter count alone is insufficient to predict memorization capacity; architectural structure, particularly depth, plays a critical role in determining achievable storage efficiency.

6 Discussion

Capacity as an emergent property of architecture and optimization. Our results show that memorization capacity is not determined solely by parameter count. While increasing P expands the

potential storage budget, the achievable capacity N_{crit} depends strongly on how those parameters are organized and whether optimization can successfully utilize them.

Across all experiments, we observe that capacity is best interpreted as an *upper-bound property under favorable optimization conditions*, rather than a deterministic function of architecture alone. This motivates our use of the maximum N_{crit} across trials as an estimate of the model’s achievable storage ceiling.

Depth vs. width: anisotropic scaling. A central finding is the asymmetric effect of scaling depth and width. Increasing depth yields consistent and monotonic improvements in capacity, even as optimization variance grows. In contrast, increasing width exhibits diminishing returns and, beyond a threshold, leads to degraded performance and instability.

This indicates that parameter count alone is insufficient to predict capacity. Instead, architectural structure governs how effectively parameters are utilized. Depth appears to provide a more reliable mechanism for increasing capacity, likely by enabling more expressive sequential transformations and more flexible routing between prefix and payload representations.

However, we emphasize that this result reflects *realized capacity under standard optimization*, not necessarily the true representational limit of the architecture. The degradation observed at large widths is likely driven by optimization difficulty rather than an intrinsic loss of capacity.

Optimization landscape and initialization sensitivity. Near the capacity boundary, training exhibits a transition regime where outcomes become highly sensitive to initialization. For the same architecture and dataset size, some runs achieve perfect memorization while others plateau slightly below. This behavior indicates the presence of multiple basins in the loss landscape, with only a subset corresponding to exact memorization solutions.

The observed variance in N_{crit} across trials further reinforces this interpretation. Capacity is not a fixed value, but a distribution induced by optimization dynamics. Initialization plays a critical role in determining whether training converges to a high-capacity solution, effectively acting as a gate on accessible storage.

This has two implications. First, reported capacity should be interpreted as a best-case estimate rather than an average outcome. Second, architectural comparisons are inherently entangled with optimization properties, as some configurations are more likely to reach favorable regions of the loss landscape.

Limits of parameter-based scaling laws. The parameter scaling analysis shows substantial dispersion in N_{crit} at fixed P , contradicting the expectation of a simple scaling law. Models with similar parameter counts can differ in capacity by an order of magnitude depending on their depth and width. This suggests that memorization capacity does not follow a universal power-law in parameter count. Instead, it depends on both parameter quantity and architectural efficiency. In particular, deeper models consistently achieve higher capacity per parameter, defining the empirical Pareto frontier in our experiments.

Interpretation as associative memory. Under our formulation, the transformer operates as an associative memory system mapping prefixes (keys) to payloads (values). From this perspective, capacity reflects the number of key–value pairs that can be reliably stored and retrieved.

The observed dependence on depth suggests that capacity is not limited purely by storage, but also by the model’s ability to implement effective routing between keys and values. This aligns with the view that attention layers act as a form of content-based addressing, where additional depth increases the complexity of representable associations.

Limitations. Several limitations should be noted. First, our definition of capacity is based on the maximum over a finite number of trials, and therefore depends on sampling over random initializations. Second, the observed degradation at large widths may reflect optimization failures rather than true capacity limits. Third, our experiments use synthetic, high-entropy data; real-world data with structure may exhibit different scaling behavior.

Summary. Overall, our results indicate that memorization capacity in decoder-only transformers is jointly determined by parameter count, architectural structure, and optimization dynamics. Depth provides a more effective scaling axis than width under standard training, while initialization and optimization play a critical role in determining whether theoretical capacity is realized in practice.

7 Future work

Our findings illuminate several promising avenues for future research into transformer memorization and capacity scaling.

Isolating theoretical width capacity. A central open question is whether the observed collapse at large embedding widths reflects a fundamental architectural limit or an optimization bottleneck. Future experiments could employ width-aware optimization frameworks, such as Maximal Update Parametrization (μP), to dynamically scale initialization variances and learning rates relative to d_{model} . Stabilizing gradient dynamics in wide models would clarify whether high-width architectures can achieve greater memorization capacity under ideal optimization conditions.

Structured and compressible data. Our study used uniform, high-entropy synthetic sequences to probe raw parameter storage. An important extension is to introduce structured or compressible data distributions, such as Zipfian token frequencies or hidden Markov dependencies. This would reveal how models allocate parameters when redundancy exists, bridging the gap between theoretical capacity bounds and practical memorization in natural datasets.

Disentangling capacity from optimization. Even under favorable initialization, capacity estimates are influenced by optimization dynamics. Future work could improve optimization procedures—via alternative initializations, adaptive curricula, or advanced optimizers—to reduce variance and more reliably approach the architecture’s true storage ceiling.

Scaling beyond current regimes. Our experiments were limited to small-to-medium models and short sequences. Extending the analysis to larger transformers and longer sequences would test whether the depth advantage and width limitations persist, and whether the observed transition and instability regimes scale predictably.

Alternative architectural factors. While this study focused on depth and embedding dimension, other design choices—attention head count, feed-forward expansion ratio, positional encoding schemes, and normalization techniques—may impact both capacity and optimization stability. Exploring these factors could uncover additional axes for efficient memorization.

Mechanistic analysis of memorization. While our work measures capacity at a system level, the internal mechanisms remain unclear. Future studies could analyze learned representations and attention patterns to understand how key–value associations are encoded and retrieved, and how depth facilitates routing.

Initialization and training dynamics. Capacity is highly sensitive to initialization, particularly near structural limits. Investigating how initialization schemes, loss landscape geometry, and convergence behavior influence high-capacity solutions could lead to methods that more consistently realize the full potential of a given architecture.

Toward theoretical models of capacity. Finally, developing theoretical models linking architectural parameters, data entropy, and optimization dynamics could provide predictive insights. Integrating information-theoretic bounds with practical optimization constraints remains an open challenge.

References

- Viola Folli, Marco Leonetti, and Giancarlo Ruocco. On the maximum storage capacity of the hopfield model. *Frontiers in Computational Neuroscience*, Volume 10 - 2016, 2017. ISSN 1662-5188. doi: 10.3389/fncom.2016.00144. URL <https://www.frontiersin.org/journals/computational-neuroscience/articles/10.3389/fncom.2016.00144>.
- J J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982. doi: 10.1073/pnas.79.8.2554. URL <https://www.pnas.org/doi/abs/10.1073/pnas.79.8.2554>.

Dmitry Krotov and John J. Hopfield. Dense associative memory is robust to adversarial inputs. *CoRR*, abs/1701.00939, 2017. URL <http://arxiv.org/abs/1701.00939>.

Xingyu Lu, Xiaonan Li, Qinyuan Cheng, Kai Ding, Xuanjing Huang, and Xipeng Qiu. Scaling laws for fact memorization of large language models, 2024. URL <https://arxiv.org/abs/2406.15720>.